

Problem Statement 1 – Banking

Your task is to **wrap this database with microservices** (minimum four) using proper service boundaries and database-per-service design and **no shared tables**. Where a table needs to be accessed by more than one service, you must **design for decoupling** (e.g., denormalize/ copy/ replicate what's needed) rather than using a single shared DB or cross-DB joins.

You are provided a **Banking Transactions Database** with the following base tables and sample data:

- **Customers** - (customer_id, name, email, phone, kyc_status, created_at)
- **Accounts** - (account_id, customer_id, account_number, account_type, balance, currency, status, created_at)
- **Transactions** - (txn_id, account_id, amount, txn_type, counterparty, reference, created_at)

Sample data (60 customers / ~90 accounts / 300 txns) has been provided to speed up local dev.

1) Develop a Microservices-Based Application (≥ 4 services; each in its own repo) (6 Marks)

Create at least **four** microservices, each maintained in a **separate GitHub repository**:

1. Customer Service

- CRUD for customer profiles and KYC status.

2. Account Service

- Create/close accounts, fetch balances, change status.

3. Transaction Service

- Record deposits, withdrawals, and transfers; fetch statements.
- Business rules (examples):
 - No overdraft for basic accounts.
 - All transfers must create two entries (debit/credit).
 - Idempotency key support for /transfer.

4. Notification Service

- Sends email/SMS notifications for high-value transactions and account status changes.

Service Constraints & Boundaries

- **Database-per-Service:** each service owns its schema (no shared tables). Replicate minimal fields as needed for queries.
- **API Contracts:** publish Swagger/OpenAPI specs for each service.
- **Config:** use environment variables or ConfigMap/Secrets for DB credentials and SMTP keys.

2) Database Design and Pattern (1.5 Marks)

- Start from the **given base schema** and refine into **service-owned schemas**, e.g.:
 - ✓ **Customer DB:** customers
 - ✓ **Account DB:** accounts (plus a read-optimized projection of customer_id, name)
 - ✓ **Transaction DB:** transactions + idempotency table
 - ✓ **Notification DB:** notifications_log
- Show an **ER diagram per service** and a **context map** showing data ownership and replicated read models.
- Implement basic **CRUD + integrity** (FKs where applicable inside a service boundary).

Please Note: Seed data for local development is provided in CSV. Students may extend the dataset or add new features/entities as per their own design requirements, provided the original base schema and relations remain usable.

Additional business rules:

Daily transfer limit (e.g., ₹2,00,000) – enforce & expose clear error.

Frozen accounts cannot transact.

Currency: assume single currency (e.g., INR) unless you design an explicit FX flow.

3) Inter-Service Communication (Low Coupling) (2.5 Marks)

- **Primary mode: Synchronous REST** (or gRPC) between services.
 - ✓ Example workflow:
 1. Transaction Service receives /transfer.
 2. It calls Account Service to verify source balance and lock funds.
 3. Writes debit/credit entries; unlocks funds; returns confirmation.
 4. Calls Notification Service to send alert.
- **Asynchronous events** (e.g., “TransactionCreated”) to Notification Service.
 - ✓ Kafka/RabbitMQ, **choose REST callbacks or polling** instead.

- Implement **resilience**: timeouts, retries (with backoff), and idempotency for /transfer.

4) Containerization with Docker (2 Marks)

- **Write a Dockerfile** for each service.
- Provide a **docker-compose** (for local multi-service bring-up), including individual service containers + their DBs.
- Verify service networking and DB connectivity.
- **Screenshots**: docker ps, service health endpoints, sample API calls.

5) Deployment on Minikube (Local Kubernetes) (2 Marks)

- Create **Kubernetes manifests** for each service:
 - ✓ **Deployment** (probes, resource requests/limits).
 - ✓ **Service** (ClusterIP; NodePort/Ingress for an API gateway if used).
 - ✓ **ConfigMap/Secret** for configuration and credentials.
 - ✓ **PersistentVolumeClaim** for databases (or run managed DB containers with PVCs).
- Demonstrate **end-to-end flow** inside the cluster:
 - ✓ Create customer → open account → make transfer → see notification logs.
- **Screenshots**: kubectl get pods, kubectl get svc, logs (kubectl logs), and a working curl/Postman run against NodePort/Ingress.

6) Monitoring Tasks (2 Marks)

Observability (Golden Signals) Sample given for reference only.

Metrics:

RED/USE: request **Rate, Errors, Duration** (histograms).

Business: transactions_total, failed_transfers_total,
balance_check_latency_ms.

Logs: structured JSON; include correlationId/traceId, path, latency; **mask PII** (email/phone).

Dashboards: latency p50/p90/p99, error rate, RPS; include a trace screenshot for the transfer flow.

7) Documentation (2 Marks)